

Stop Using Exact Match! Master Embeddings & Vector Search

The Secret Behind AI-Powered Semantic Search

By Dipanshu Simejiya

What You'll Learn

Topic	Description
 Traditional vs Vector DB	Why exact match fails
 Embeddings	Text → Numbers
 Cosine Similarity	Measure similarity
 HuggingFace	Free & Open Source
 OpenAI	Production-ready API
 Semantic Search	Find by meaning

Traditional DB vs Vector DB

✗ Traditional Database (SQL)

```
SELECT * FROM animals WHERE name = 'cat'
```

Problems:

- ✗ Exact match only
- ✗ Search "cat" → returns only "cat"
- ✗ Misses: kitten, feline, kitty
- ✗ No understanding of meaning

✓ Vector Database

```
# Search by meaning, not just text
query = "cat"
results = ["cat", "kitten", "feline"] # Finds similar meanings!
```

Benefits:

-  Semantic search
-  Search "cat" → finds kitten, feline too
-  Understands: kitten $\approx$ cat
-  Captures relationships

 **Key Insight:** Vector DBs find **similar meanings**, not just exact text matches!



What are Embeddings?

Embeddings convert text into numbers (vectors) that capture meaning.

```
"Apple" → [0.2, 0.3, 0.5, ...]
"Banana" → [0.25, 0.35, 0.45, ...] ← Similar vectors!
"Car" → [-0.5, 0.1, 0.8, ...] ← Different vector
```

Simple 2D Example

```
word_embeddings = {
    "cat": [0.8, 0.6],
    "kitten": [0.75, 0.65], # Very close to cat!
    "dog": [0.7, 0.3],
    "car": [-0.5, 0.2] # Completely different
}
```

 Real embeddings have **384 to 3072 dimensions** for better accuracy!



Real World Example: Netflix

How Netflix recommends movies using embeddings:

Movie	Action	Comedy	Suspense
 Iron Man	0.95	0.2	0.6
 Hulk	0.96	0.4	0.7
 Sherlock	0.6	0.75	0.9

If you liked Iron Man → Netflix recommends Hulk

Why? Both have high Action scores = Similar vectors = Similar recommendations!

💡 This is how recommendation systems work!



Understanding Embedding Dimensions

What does it mean by 384, 768, or 3072 dimensions?

Each dimension is like a feature that captures a different aspect of meaning. More dimensions = more nuanced understanding of text.

Visual Analogy

2D (x, y):	Describes position on a flat surface
3D (x, y, z):	Adds depth – more accurate representation
384D:	Captures 384 different aspects of meaning!
3072D:	Even more nuanced understanding

Dimensions Comparison

Dimensions	Storage per Vector	Speed	Quality	Best For
384	~1.5 KB	⚡ Fastest	Good	Most applications, fast retrieval
768	~3 KB	🚀 Fast	Better	Balanced performance & quality
1536	~6 KB	🏃 Medium	Great	Production systems
3072	~12 KB	🐢 Slower	Best	Maximum accuracy needed

Pros & Cons

Lower Dimensions (384-768)

✅ Pros:

- Faster computation and search
- Less storage space required
- Lower memory usage
- Perfect for real-time applications

- Cost-effective for large datasets

✗ Cons:

- Less nuanced understanding
- May miss subtle differences
- Lower accuracy on complex queries

Higher Dimensions (1536-3072)

✓ Pros:

- Captures subtle semantic differences
- Better accuracy on complex queries
- More nuanced understanding of context
- Superior for multilingual tasks

✗ Cons:

- Slower computation time
- More storage space needed
- Higher memory requirements
- More expensive to process

Example: Same Text, Different Dimensions

```
text = "The quick brown fox jumps over the lazy dog"

# 384 dimensions
[0.23, -0.15, 0.67, ..., 0.42] # 384 numbers

# 1536 dimensions
[0.23, -0.15, 0.67, ..., 0.42, 0.88, -0.31, ..., 0.19] # 1536 numbers

# 3072 dimensions
[0.23, -0.15, 0.67, ..., 0.42, 0.88, -0.31, ..., 0.19, ...] # 3072 numbers
```

💡 **Rule of Thumb:** Start with 384-768 dimensions. Only move to higher dimensions if you need that extra accuracy and can afford the computational cost!



Cosine Similarity

Formula to measure how similar two vectors are:

$$\text{Cosine Similarity} = (A \cdot B) / (||A|| \times ||B||)$$

Score Meaning

Score	Meaning
1.0	Identical
0.5	Somewhat similar
0.0	Completely different

Python Implementation

```
import numpy as np

def cosine_similarity(vec1, vec2):
    """
    Calculate similarity between two vectors.
    Returns: 0 (different) to 1 (identical)
    """
    dot_product = np.dot(vec1, vec2)
    norm_a = np.linalg.norm(vec1)
    norm_b = np.linalg.norm(vec2)
    return dot_product / (norm_a * norm_b)

# Example usage
cat = [0.8, 0.6]
kitten = [0.75, 0.65]
car = [-0.5, 0.2]

print(cosine_similarity(cat, kitten)) # 0.99 - Very similar!
print(cosine_similarity(cat, car))    # 0.12 - Very different!
```

HuggingFace Embeddings

Free & Open Source | Runs Locally | No API Key Required

Installation

```
pip install langchain-huggingface sentence-transformers
```

Complete Code Example

```
from langchain_huggingface import HuggingFaceEmbeddings

# Initialize the model (downloads automatically, runs locally)
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

# Single text → Single vector (384 dimensions)
text = "I love machine learning!"
vector = embeddings.embed_query(text)
print(f"Vector length: {len(vector)}") # 384

# Multiple texts → Multiple vectors
documents = [
    "The cat sat on the mat",
    "A feline rested on the rug",
    "Python is a programming language"
]
vectors = embeddings.embed_documents(documents)
print(f"Number of vectors: {len(vectors)}") # 3
```

Popular HuggingFace Models

Model	Dimensions	Speed	Best For
all-MiniLM-L6-v2 ⭐	384	Fast	General purpose
all-mpnet-base-v2	768	Medium	Best quality
multi-qa-MiniLM-L6-cos-v1	384	Fast	Q&A systems
paraphrase-multilingual-MiniLM-L12-v2	384	Medium	50+ languages

💡 **Recommended:** Start with all-MiniLM-L6-v2 - fast, free, and good quality!

OpenAI Embeddings

High Quality | Production Ready | Paid API

Installation

```
pip install langchain-openai
```

Complete Code Example

```
import os
from langchain_openai import OpenAIEmbeddings

# Set your API key
os.environ["OPENAI_API_KEY"] = "your-api-key-here"

# Initialize OpenAI embeddings
embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small" # 1536 dimensions
)

# Single text → Single vector
text = "LangChain is amazing for building AI apps!"
vector = embeddings.embed_query(text)
print(f"Vector length: {len(vector)}") # 1536

# Multiple texts → Multiple vectors
documents = [
    "Python is a programming language",
    "LangChain helps build LLM applications",
    "Embeddings convert text to numbers"
]
vectors = embeddings.embed_documents(documents)
```

OpenAI Models Comparison

Model	Dimensions	Cost	Best For
text-embedding-3-small 🌟	1536	\$0.02/1M tokens	Cost-effective
text-embedding-3-large	3072	\$0.13/1M tokens	Highest quality
text-embedding-ada-002	1536	\$0.10/1M tokens	Legacy

💡 **Recommended:** Use text-embedding-3-small for most use cases!

Similarity Comparison Example

Let's compare sentences and see their similarity scores:

```
from langchain_openai import OpenAIEmbeddings
import numpy as np
```

```

def cosine_similarity(vec1, vec2):
    return np.dot(vec1, vec2) / (np.linalg.norm(vec1) * np.linalg.norm(vec2))

# Initialize embeddings
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

# Our test sentences
sentences = [
    "The cat sat on the mat",
    "A feline rested on the rug",
    "I love programming in Python",
    "Python is my favorite language"
]

# Get embeddings for all sentences
vectors = embeddings.embed_documents(sentences)

# Compare similarities
print("Similarity Results:")
print("-" * 50)
print(f"'cat on mat' vs 'feline on rug': {cosine_similarity(vectors[0], vectors[1])}")
# Output: 0.655 ← Similar meaning!

print(f"'Python programming' vs 'favorite language': {cosine_similarity(vectors[2], vectors[3])}")
# Output: 0.708 ← Similar meaning!

print(f"'cat on mat' vs 'Python programming': {cosine_similarity(vectors[0], vectors[2])}")
# Output: 0.089 ← Very different!

```

Results Explained

Comparison	Score	Meaning
cat + mat vs feline + rug	0.655	✅ Similar (both about cats)
Python love vs Python favorite	0.708	✅ Similar (both about Python)
cat + mat vs Python	0.089	❌ Different topics

Building Semantic Search

The complete semantic search function:


```

import numpy as np
from langchain_openai import OpenAIEmbeddings

def cosine_similarity(vec1, vec2):
    """Calculate similarity between two vectors (0 to 1)"""
    return np.dot(vec1, vec2) / (np.linalg.norm(vec1) * np.linalg.norm(vec2))

def semantic_search(query, documents, model, top_k=3):
    """
    Find the most relevant documents for a query.

    Args:
        query: The search query string
        documents: List of documents to search through
        model: Embedding model instance
        top_k: Number of top results to return

    Returns:
        List of (score, document) tuples, sorted by relevance
    """
    # Step 1: Convert query to vector
    query_embedding = model.embed_query(query)

    # Step 2: Convert all documents to vectors
    doc_embeddings = model.embed_documents(documents)

    # Step 3: Calculate similarity for each document
    similarities = []
    for i, doc_emb in enumerate(doc_embeddings):
        score = cosine_similarity(query_embedding, doc_emb)
        similarities.append((score, documents[i]))

    # Step 4: Sort by similarity (highest first)
    similarities.sort(reverse=True, key=lambda x: x[0])

    # Step 5: Return top results
    return similarities[:top_k]

# Example usage
documents = [
    "LangChain is a framework for building LLM applications",
    "Python is a high-level programming language",
    "Machine learning is a subset of artificial intelligence",

```

```

    "Embeddings convert text into numerical vectors",
    "The weather today is sunny and warm"
]

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

# Search!
query = "What is LangChain?"
results = semantic_search(query, documents, embeddings)

print(f"Query: '{query}'")
print("-" * 50)
for score, doc in results:
    print(f"Score: {score:.3f} | {doc}")

# Output:
# Score: 0.676 | LangChain is a framework for building LLM applicati
# Score: 0.130 | Python is a high-level programming language
# Score: 0.101 | Embeddings convert text into numerical vectors

```

💡 This is the foundation of RAG (Retrieval Augmented Generation)!

📄 Quick Reference Cheatsheet

😊 HuggingFace vs 🧙 OpenAI

Feature	HuggingFace	OpenAI
Cost	Free	Paid API
Location	Runs locally	Cloud-based
Default Model	all-MiniLM-L6-v2	text-embedding-3-small
Dimensions	384-768	1536-3072
API Key	Not required	Required
Best For	Learning, Privacy	Production

Key Methods

Method	Input	Output
<code>embed_query(text)</code>	Single string	Single vector
<code>embed_documents(list)</code>	List of strings	List of vectors

cosine_similarity(a, b) Two vectors Score (0 to 1)

Quick Start Code

```
# HuggingFace (Free)
from langchain_huggingface import HuggingFaceEmbeddings
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers

# OpenAI (Paid)
from langchain_openai import OpenAIEmbeddings
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

# Use either one the same way!
vector = embeddings.embed_query("Your text here")
vectors = embeddings.embed_documents(["Text 1", "Text 2"])
```

✅ What You've Learned

- ✅ **Traditional vs Vector DB** - Why semantic search beats exact match
- ✅ **Embeddings** - Converting text to meaningful numbers
- ✅ **Cosine Similarity** - Measuring how similar vectors are
- ✅ **HuggingFace** - Free, open-source, local embeddings
- ✅ **OpenAI** - Production-ready, high-quality embeddings
- ✅ **Semantic Search** - Finding relevant documents by meaning

🎯 Key Takeaway

Embeddings convert text into numbers that capture meaning, enabling semantic search that understands context, not just keywords.

👋 Follow for More!

Dipanshu Simejiya

🔗 Follow me for more **Gen AI**, **RAG**, and **LangChain** content!

🔄 *Repost if this helped you!*